

VITyarthi - Build Your Own Project

EXAMPREP

Multi-Subject Question Bank & Practice System

Project Report

Course: Java Programming

Student Name: naman gupta

Registration No.: 25BAI11448

Faculty / Mentor: sandeep monga

Academic Year: 2026

Table of Contents

1. 1. Introduction
2. 2. Problem Statement
3. 3. Objectives
4. 4. Scope and Target Users
5. 5. Functional Requirements
6. 6. Non-Functional Requirements
7. 7. System Architecture
8. 8. System Design Diagrams
9. 9. Design Decisions and Rationale
10. 10. Implementation Details
11. 11. Data and Storage Design
12. 12. Screenshots / Results
13. 13. Testing Approach
14. 14. Challenges Faced
15. 15. Learnings and Key Takeaways
16. 16. Future Enhancements
17. 17. Conclusion
18. 18. References

1. Introduction

ExamPrep is a console-based Java application developed as a multi-subject question bank and practice system. It brings question browsing, randomized MCQ practice, score recording, wrong-question revision, and basic question administration into one application.

The application is designed around a simple learning loop: a student selects a subject and module, attempts questions, receives a score, and can later revisit incorrectly answered questions. Questions are organized by subject, module, and difficulty so that practice can be targeted to a specific part of a syllabus.

The current question bank contains 800 questions: 200 for Java Programming, 200 for Operating Systems, 200 for Discrete Mathematics, and 200 for Digital Logic and Computer Architecture.

2. Problem Statement

Students preparing for examinations frequently switch between separate sources for practice questions, results, and revision. This makes it difficult to maintain a structured practice workflow and track mistakes.

ExamPrep addresses this problem by providing a unified application that stores a structured question bank, generates practice tests, records performance, and preserves incorrectly answered question IDs for later revision.

3. Objectives

- Provide a structured question bank aligned with the supplied academic syllabi.
- Allow users to filter questions by subject, module, and difficulty.
- Generate randomized practice tests from selected subject and module combinations.
- Calculate and store test results.
- Store incorrectly answered question IDs for revision.
- Provide basic administrative operations for adding, viewing, and deleting questions.
- Demonstrate Java concepts including classes, packages, collections, file handling, exceptions, and modular design.

4. Scope and Target Users

4.1 Scope

The project covers a menu-driven console application with five major functional modules: Question Bank, Practice Test, Results & Performance, Revision, and Admin / Question Management. Persistent information is stored in plain text files, so the application does not depend on a database server.

4.2 Target Users

- College students preparing for examinations.
- An administrator or question-bank manager responsible for maintaining questions.

5. Functional Requirements

ID	Requirement	Description
FR1	Question Bank	View all questions and filter them by subject, module, or difficulty.
FR2	Practice Test	Select subject, module, question count, attempt randomized MCQs, and calculate a score.
FR3	Results & Performance	Save and display previous test results, correct counts, and percentage scores.
FR4	Revision	Store incorrectly answered question IDs and display those questions with correct answers.
FR5	Admin / Question Management	Authenticate the administrator and allow adding, viewing, and deleting questions.
FR6	File Persistence	Read and write questions, results, and wrong-question IDs using text files.

6. Non-Functional Requirements

Requirement	Implementation
Usability	The interface is menu-driven and uses simple text prompts.
Reliability	Invalid input and file errors are handled through validation and exceptions.
Maintainability	Responsibilities are separated across seven Java classes and package structures.
Performance	Questions are loaded into ArrayList and filtered in memory for fast access at the intended project scale.
Resource Efficiency	The application uses standard Java libraries and lightweight text files.
Security	Administrative question-management functions are protected by a password check.

7. System Architecture

The system follows a simple layered organization. Main.java provides the console entry point and routes menu choices. Core services implement question management, quiz execution, and results/revision functions. FileManager.java provides the file access layer, while the text files provide persistent storage.

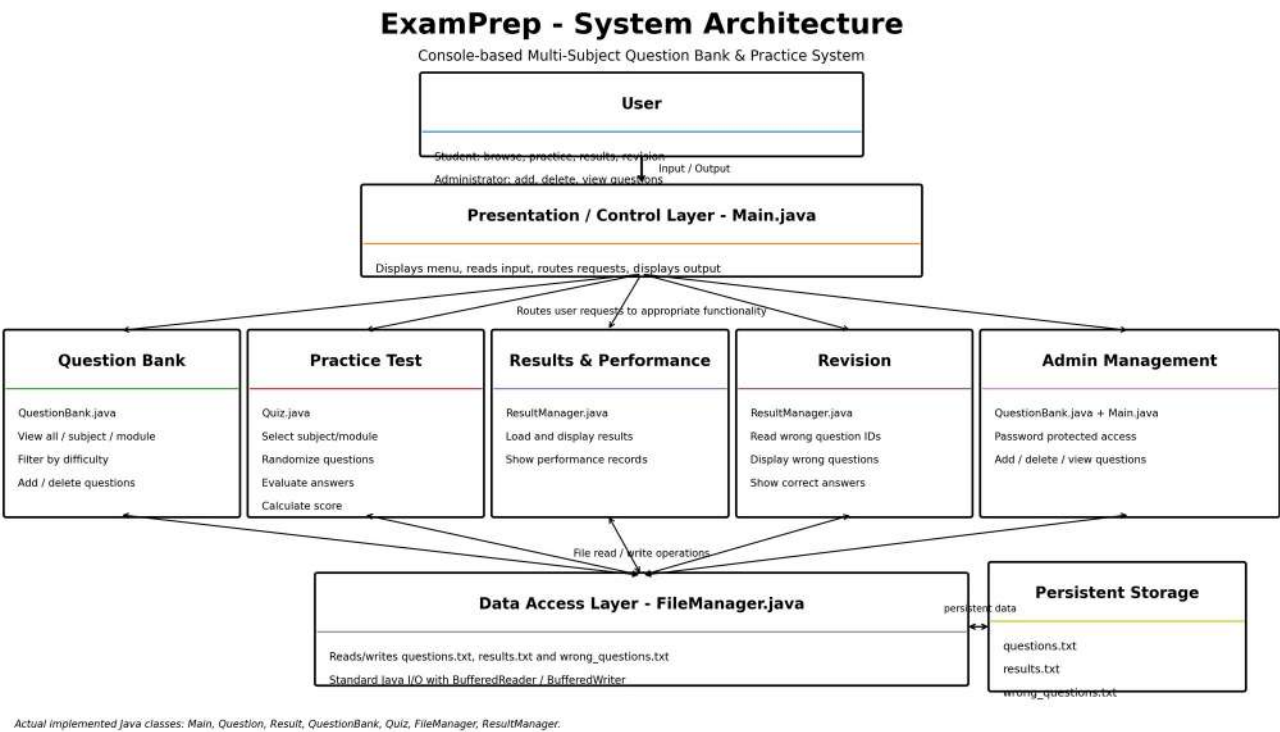


Figure 1. System Architecture Diagram

The architecture is intentionally lightweight: it demonstrates modular separation without introducing external frameworks or a database dependency.

8. System Design Diagrams

8.1 Process Flow / Workflow

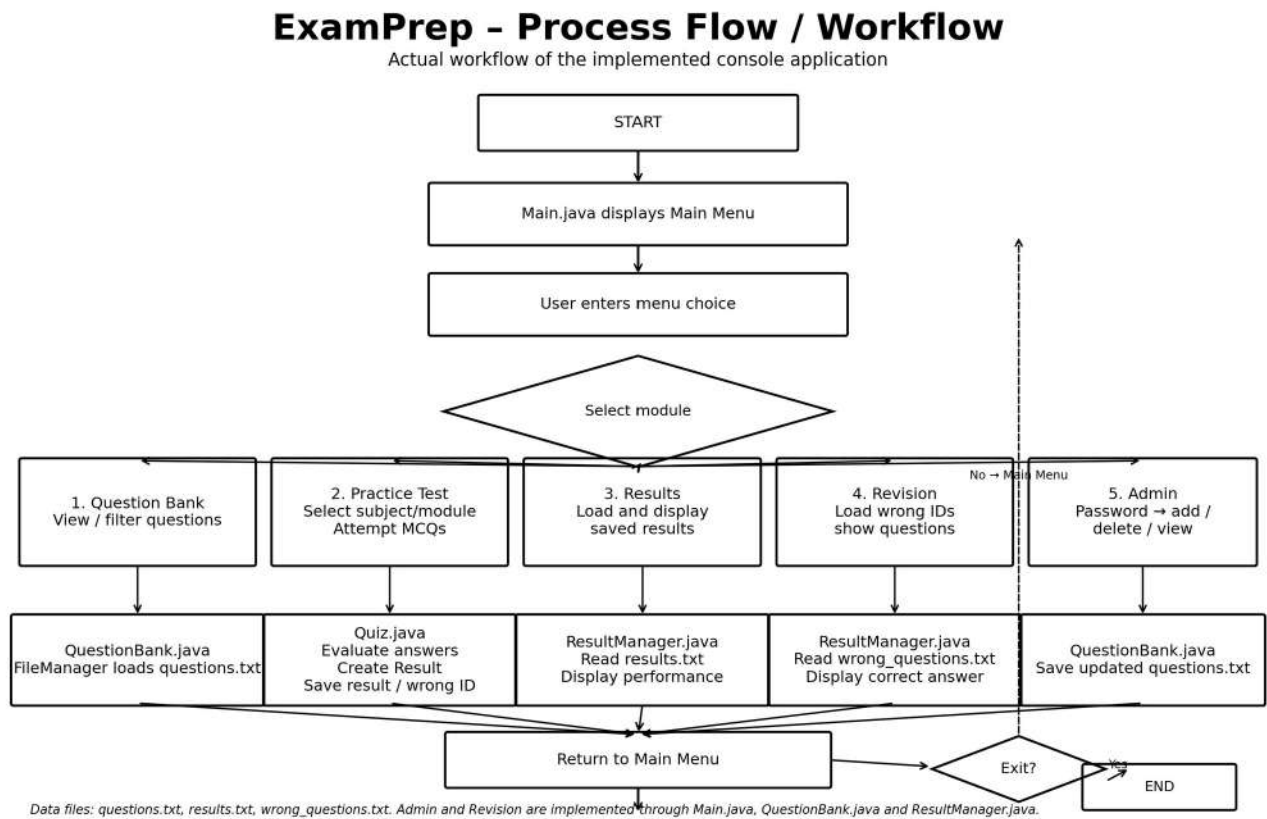


Figure 2. Process Flow / Workflow Diagram

8.2 Use Case Diagram

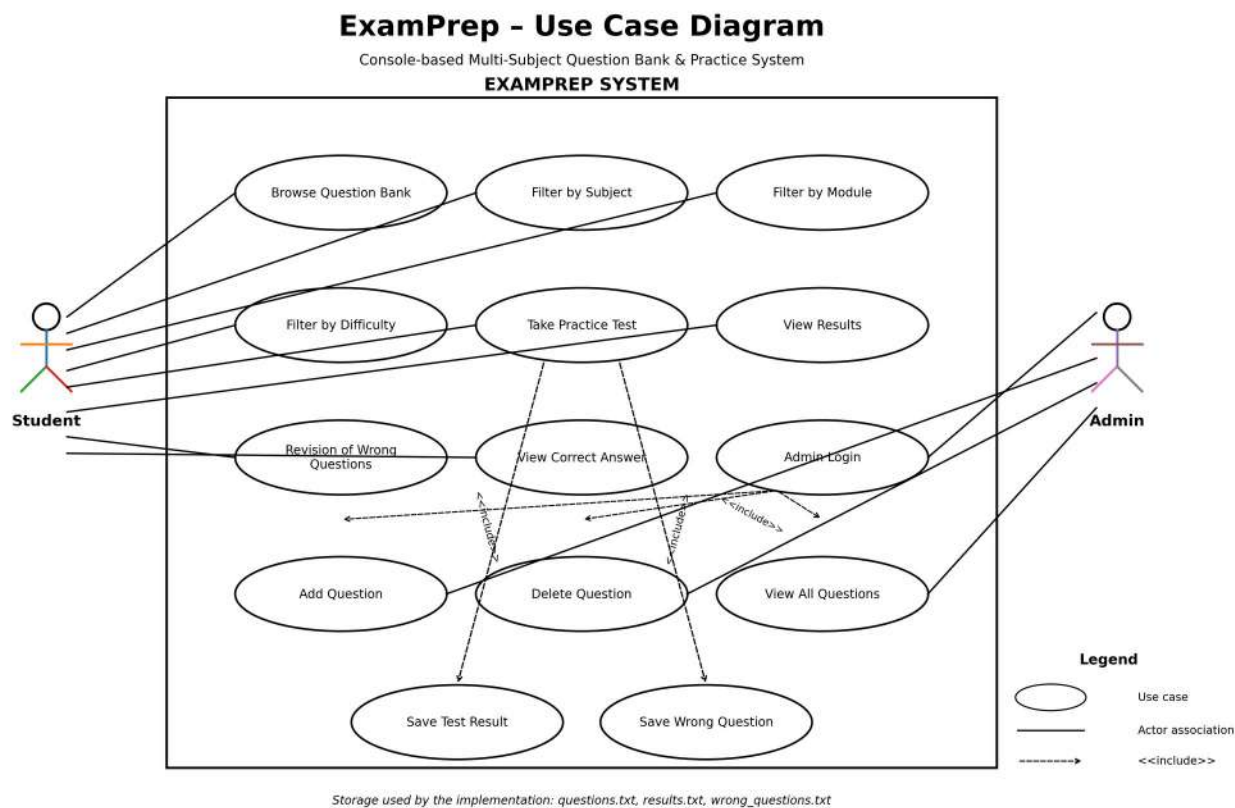
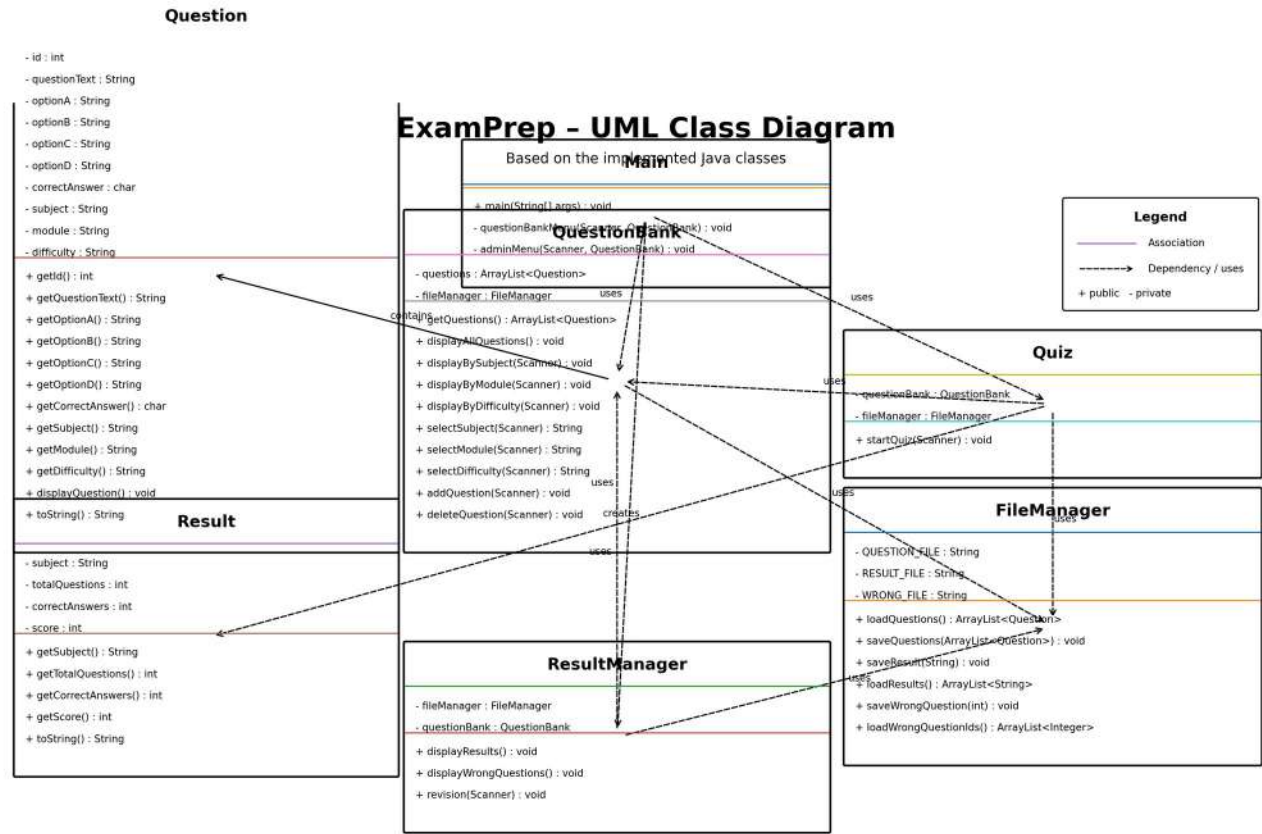


Figure 3. Use Case Diagram

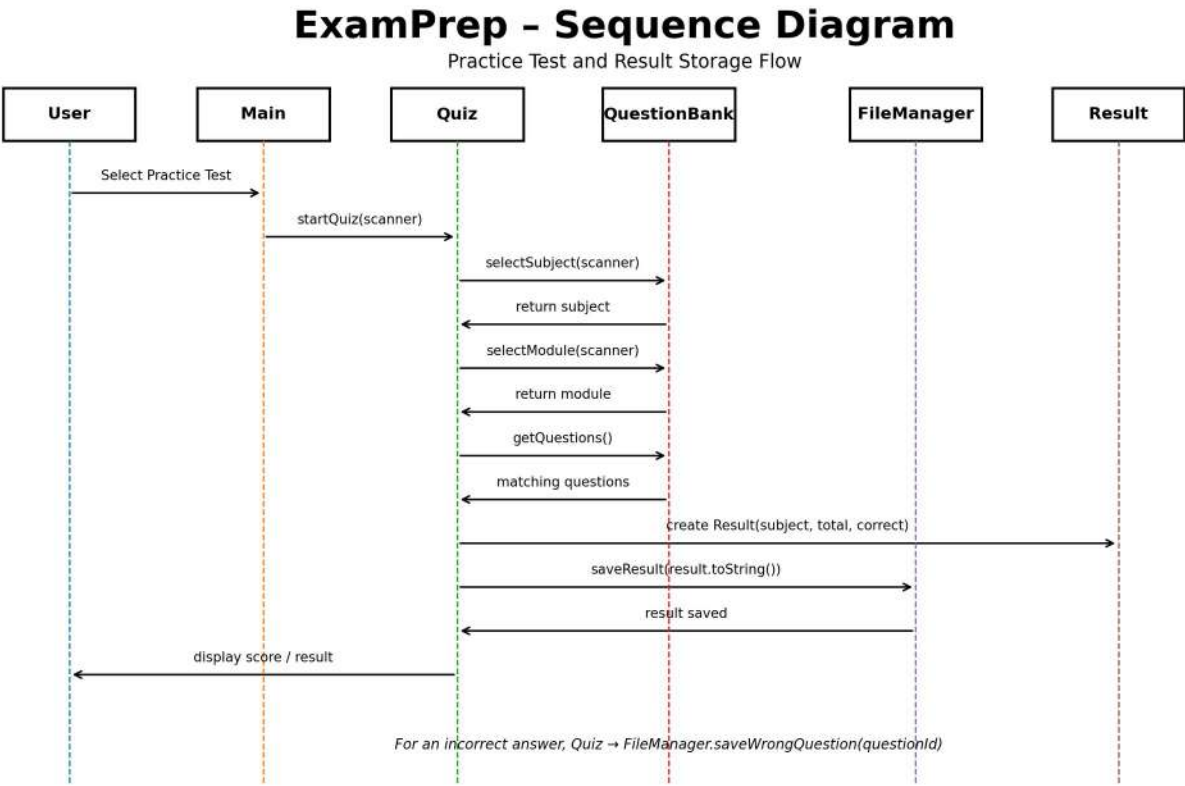
8.3 Class Diagram



Only the 7 classes implemented in the project are shown. Admin and Revision are methods/functions of existing classes, not separate classes.

Figure 4. UML Class Diagram

8.4 Sequence Diagram



The diagram follows the current implementation and shows the main Practice Test interaction; QuestionBank internally uses FileManager to load the question bank.

Figure 5. Practice Test Sequence Diagram

9. Design Decisions and Rationale

Java packages: The code is divided into model and service packages to separate data objects from application services.

ArrayList: An ArrayList is used for the in-memory question bank because questions are frequently filtered and iterated.

Text-file storage: Plain text files keep the project self-contained, easy to run, and free from database configuration.

Delimiter-based records: questions.txt uses ~ as the record-field separator, while results.txt uses |; this makes file parsing straightforward.

Randomization: Collections.shuffle() is used so practice tests can present questions in a different order.

Exception handling: Input parsing and file I/O are guarded against invalid values and common I/O failures.

Modular service classes: QuestionBank, Quiz, FileManager, ResultManager, and the model classes give each major responsibility a clear location.

10. Implementation Details

10.1 Project Structure

```
JAVAPROJECTVITYARTHI/
|
|-- src/
|   |-- Main.java
|   |
|   |-- model/
|   |   |-- Question.java
|   |   |-- Result.java
|   |
|   |-- service/
|   |   |-- FileManager.java
|   |   |-- QuestionBank.java
|   |   |-- Quiz.java
|   |   |-- ResultManager.java
|   |
|-- data/
|   |-- questions.txt
|   |-- results.txt
|   |-- wrong_questions.txt
|
|-- docs/
|   |-- architecture.png
|   |-- workflow.png
|   |-- use_case.png
|   |-- class_diagram.png
|   |-- sequence_diagram.png
|   |-- storage_design.png
|
|-- README.md
|-- statement.md
```

10.2 Main Classes

Class	Responsibility
Main	Starts the application, displays the main menu, and routes user choices.
Question	Represents one MCQ with its ID, text, four options, correct answer, subject, module, and difficulty.
Result	Represents a completed test result and calculates the percentage score.
QuestionBank	Loads and manages questions, filtering, adding, and deleting question records.
Quiz	Runs practice tests, randomizes questions, evaluates answers, and records wrong question IDs and results.
FileManager	Performs all question, result, and wrong-question file I/O.
ResultManager	Displays saved results and supports revision using stored wrong-question IDs.

10.3 Question Bank

The final question bank contains 800 MCQs. Each stored record contains ten fields: ID, question text, four options, correct answer, subject, module, and difficulty. The question bank is organized so the user can practice a specific subject and module instead of receiving unrelated questions.

11. Data and Storage Design

The application uses three text files for persistent storage. This satisfies the project's need for persistence without requiring a relational database.

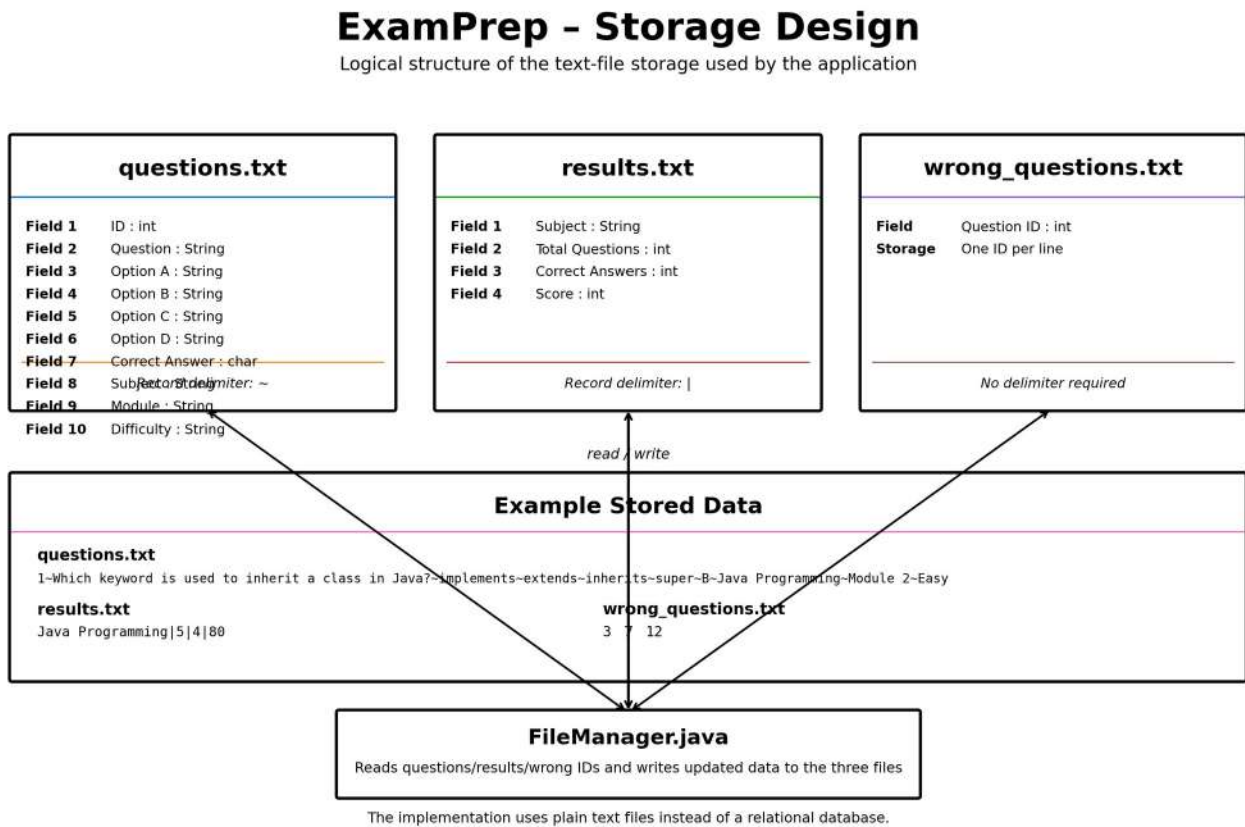


Figure 6. Storage Design

11.1 Storage Files

- **questions.txt**: stores the complete question bank using ~ between fields.
- **results.txt**: stores test result fields using | between values.
- **wrong_questions.txt**: stores one incorrectly answered question ID per line.

An ER diagram is not used because the implementation does not use a relational database. The logical storage structure is documented instead.

12. Screenshots / Results

The following outputs were verified during functional testing of the application.

Tested Area	Observed Result
Application startup	Questions loaded: 800
Practice test	A Java Programming practice test was completed with 1 question and produced Correct: 1 and Score: 100%.
Results module	The completed test appeared under Previous Results with the corresponding subject, question count, correct count, and score.
Revision module	After deliberately submitting an incorrect answer, the question ID was stored and the Revision module displayed the question and its correct answer.
Admin module	Password-protected administration displayed the Add Question, Delete Question, View All Questions, and Back options.

Representative console screenshots can be added to this section before final portal submission. The report already documents the verified outputs and the corresponding design artefacts.

13. Testing Approach

Testing was performed as functional validation through the console interface after compilation. The main workflow was exercised module by module, including both successful and incorrect-answer paths.

ID	Test	Expected	Status
T01	Compile all source files	No compilation errors	Pass
T02	Start application	Main menu displayed; 800 questions loaded	Pass
T03	View Question Bank	Questions displayed and filtering menu worked	Pass
T04	Practice Test	Question accepted, score calculated, result saved	Pass
T05	Results	Saved result displayed correctly	Pass
T06	Revision	Wrong question displayed with correct answer	Pass
T07	Admin Add	Question added successfully	Pass
T08	Admin Delete	Selected question deleted successfully	Pass
T09	Input validation	Invalid numeric/choice inputs handled without immediate termination	Pass

14. Challenges Faced

- Maintaining a consistent delimiter-based data format across a large question bank.
- Handling Scanner input cleanly when switching between menu selections and text entry.

- Ensuring incorrect answers are persisted so that revision remains connected to the practice-test workflow.
- Keeping class responsibilities separated while maintaining simple interaction between the modules.
- Managing project files, compilation, Git commits, and GitHub repository structure.

15. Learnings and Key Takeaways

- Practical use of classes, constructors, encapsulation, methods, and packages.
- Use of ArrayList and Collections.shuffle() for collection management and randomization.
- Practical Java file I/O using readers and writers.
- Exception handling and input validation for console applications.
- Designing a modular application with clear responsibilities.
- Using Git and GitHub for version control and project delivery.
- Translating a written project requirement into functional modules and documentation.

16. Future Enhancements

- Add a graphical user interface using JavaFX.
- Replace text-file storage with a relational database using JDBC.
- Add user accounts and user-specific progress tracking.
- Introduce timed examinations and configurable scoring rules.
- Add detailed analytics such as subject-wise accuracy and progress history.
- Provide import/export of question banks and administrator tools for bulk question management.

17. Conclusion

ExamPrep demonstrates a complete Java-based application that combines a structured academic question bank with practice, performance tracking, revision, and administrative question management. The final implementation uses seven Java classes, package-based organization, exception handling, collections, randomization, and file-based persistence.

The project also provides the design and documentation artefacts required for submission: problem statement, objectives, requirements, architecture, workflow, UML diagrams, storage design, implementation details, testing, challenges, learnings, future enhancements, and references.

18. References

19. VITyarthi - Build Your Own Project: General Project Instructions & Submission Guidelines, supplied project brief.
20. Java Programming syllabus supplied for the project, including modules on Java Basics, Object-Oriented Programming, Robustness and Concurrency, Files/Streams/Collection Framework, GUI/Database Connectivity, and Servlet.

21. Operating Systems syllabus supplied for the project, including Units I-V covering OS fundamentals, memory management, I/O and deadlocks, virtualization/cloud, and Linux/Windows case studies.
22. Discrete Mathematics and Graph Theory syllabus supplied for the project, including Modules 1-8.
23. CSA2003 Digital Logic and Computer Architecture syllabus supplied for the project, including Modules 1-6.